HLTP/x25-32 Microarchitecture Specifications

Contents

Introduction

Chapter	*Title*	*Page(s)*
0.	**Preamble**	3
0.1	Technical Terminology, Abbreviations	4

Main Part

Chapter	*Title*	*Page(s)*
1.	**Abstract Design**	5
1.1	Common CPU Specifications	6
2.	**Hardware**	7
2.1	General Processor Layout	
2.2	Busses	
2.3	Registers	
2.4	Processing Internals & Internal Registers	
2.4.1	Control Unit	
2.4.2	Arithmetic Logic Unit	
2.4.3	Memory Management Unit	
2.5	Hardware Memory Layout	
2.5.1	MMIO	
3.	**Instruction Set Architecture**	
3.1	Instruction Encoding	
3.2.1	Instructions Master Table	
3.2.2	List of Instructions	
3.2	Instruction Set Architecture (ISA)	
3.3.1	Assembly Syntax	
3.3.2	Assembly Binary Structure	
4.	**Execution**	
4.1	Fetch-Decode-Execute Cycle	
4.2	Reset / Boot Process	
4.3	Interrupts	

Ending

Chapter	*Title*	*Page(s)*
5.	**End**	
5.1	Notes, Limitations and Design Decisions regarding Emulation	

5.2	Useful resources	
5.3	Future Plans / Extensions	
5.4	Final Thoughts, Conclusion	

0: Preamble

Disclaimer: This project and all related content are **not** intended for any real world application. It is primarily intended as my (the author's) way of learning CPU design, aimed at providing educational use of computer architecture theory. Although trying to imitate real world implementations, some areas may significantly deviate from standard use/understanding; This usually comes down to either intentional design decisions or incompetence (by the author of this document) to fully comprehend architecture design.

Before you read this document, please be aware that it is assumed, that the reader has already familiarized with some basic computer knowledge and additionally ground level knowledge of low level CPU architecture design.

[Placeholder: How this project came to life]

0.1: Technical Terminology, Abbreviations

CPU	Central processing unit
CPI	Clocks per instruction (Multiplicative inverse of Instructions per clock / IPC)
ISA	Instruction set architecture
MSB	Most significant byte
LSB	Least significant byte
ISR	Interrupt service routine
IO	Input / output
MMIO	Memory-mapped IO
FDX (Cycle)	**F**etch-**D**ecode-e**X**ecute Cycle

Tab. 1: Commonly used abbreviations throughout this document

GPR / GP[X]	General-purpose register [X]
PC	Program counter (commonly also referred to as Instruction Pointer / IP)
IR / IRA / IRB	Instruction register / Instruction register A / Instruction register B
MAR	Memory address register
MDR	Memory data register
SR	Status register
CU	Control unit
ALU	Arithmetic logic unit
MMU	Memory management unit
RAM	Random access memory (sometimes just called “Memory”)
CON / TMEM	Console memory
VMEM	Video memory / Graphics memory

Tab. 1.1: Abbreviations for CPU hardware internals

1: Abstract Design

[Summary of what this chapter covers]
Inspiration mainly from other RISC implementations, x86 (esp. early designs such as 8086, and ARMv7)

1.1: Common CPU specifications

Word Size	32 bits / 4 bytes
Instruction Width	64 bits / 8 bytes (double word alignment)
Endianness	Big-endian (MSB first, LSB last)
Registers	8x GPRs (1 word), PC (1 word), IRA (1 word), IRB (1 word), SR (1 word), MAR (1 word), MDR (1 word)
Clocks per instruction (CPI)	6 (Arithmetic); 7 (LOAD, STORE), much more (CALL, INT, RET, IRET)
Architecture model	RISC
Pipelining	None
Branch prediction	None (requires Pipelining)
Parallelism	None
Cache architecture	No cache
Virtual memory support	Hardware address mapping, No dynamic remapping during execution
Execution modes	Real / direct mode only, No mode-switching instructions
Extensibility	Not implemented (No co-processing)

Tab. 2: Specifications common among different CPU architectures

2: Hardware

[Brief summary of what this chapter is about]

2.1: General Processor Layout

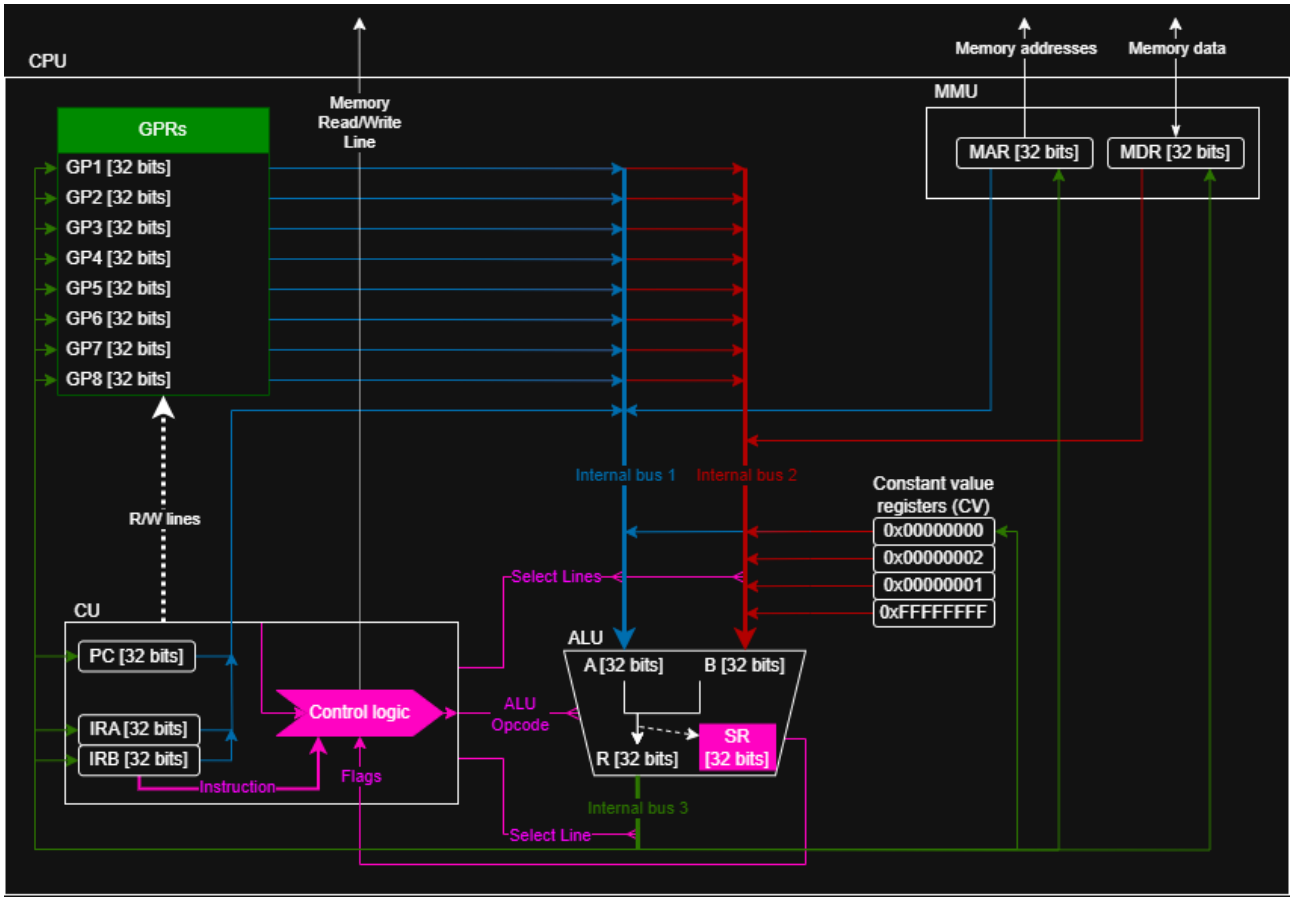

Fig. 1: The x25-32 CPU Hardware Layout as of Processor Revision 0.1-indev

Legend:
Component colors:
Pink: Elements directly affecting, processing or forwarding control signals
Blue: Direct attachments to Internal bus 1
Red: Direct attachments to Internal bus 2
Green: Direct attachments to Internal bus 3
Lines:
Dotted: Single bit line / single “wire”
Solid: Bus (32 bits)
Solid and fat: Bus (32 bits) that changes source / destination depending on control signals (using multiplexers); *Exception: “Instruction” bus from IRA + IRB to Control logic. This line is only fat to emphasize that it carries 2 words from IRA and IRB at the same time, meaning it is 64 bits wide.*
Miscellaneous:
Rounded boxes: Special purpose registers

Note: The constant value registers cannot be written to (writes are essentially ignored). The reason the ***zero*** register (0x00000000) still has an input is that it acts as a “discard” register; When the ALU’s operation has no target, this register is therefore the destination of Internal Bus 3, where the ***zero*** register is also referred to as the ***void*** register.

2.3: Registers

GPRs and Internal Registers:
Explain their function / purpose
Important: PC updated BEFORE instruction executed → This ensures the instruction can happily override the PC without additional complexity.

2.5: Hardware Memory Layout

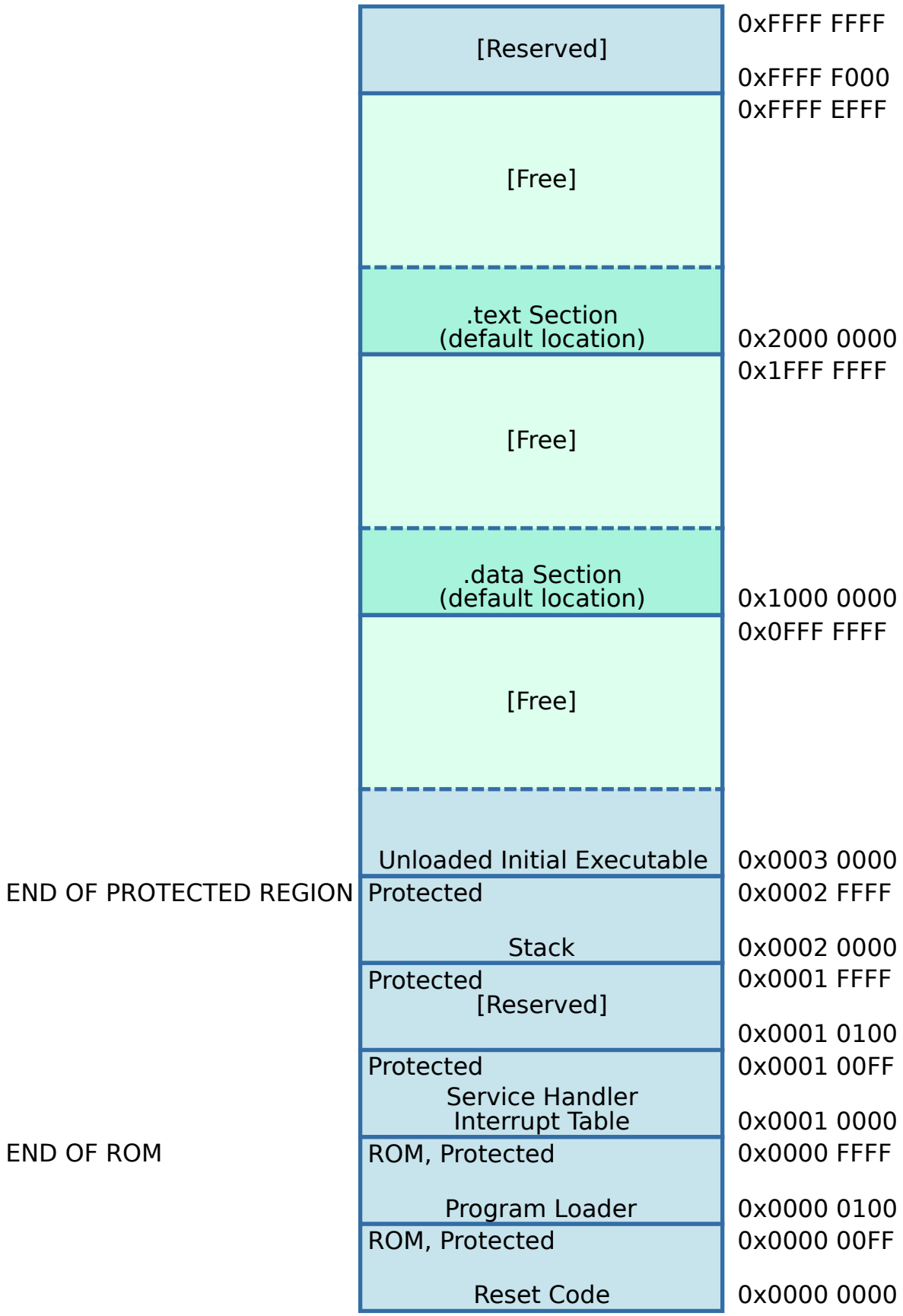

Fig. 2: The x25-32 Hardware Memory Layout as of Processor Revision 0.2-indev

This section only describes the *hardware enforced* memory layout.
The address space of the CPU is required by the CPU itself to have this structure, since it otherwise results in undefined behaviour.
Any further software-driven subdivision is possible at the discretion of the user.
The described memory layout is outlined in Fig. 2.

Legend:
Memory sections marked as ROM are on read-only devices.
Memory sections in the protected region trigger an *illegal secure memory write exception* when written to from an unprivileged point (more in about this later in this chapter).
Green sections are usually free to use by the main program, which is loaded by the program loader.

The following table explains every region in more detail:

Region name	**Explanation**
Reset Code	The reset code is a directly executable part of memory (starting at 0x00000000), which immediately runs once the CPU has experienced a hardware reset. This can either be caused by a software interrupt to the reset vector or a general hardware / software fault triggering an unrecoverable condition. The reset code brings ensures the CPU to launch in a “clean” state, e.g. clearing GPRs and calling the loader.
Program Loader	The program loader is executed once the CPU reset code has finished running. It is responsible for loading an unloaded executable from address 0x00030000 onward into its proper location to be executed. This includes, but is not limited to extracting header information, copying the .data and .text sections and appropriately modifying any address-reliant instructions in the executable to be properly aligned with the load address. When the loader is done executing, it calls the program it loaded.
Service Handler Interrupt Table (SHIT)	The Service Handler Interrupt Table (in short: SHIT) contains information about where each interrupt vector resides in actual address space. Each offset into the SHIT (interrupt vector) has a memory address, which points to the actual ISR.
[Reserved]	This memory region is reserved for later use. Do not use.
Stack	The CPU global stack is FILO (first-in-last-out) memory structure, which allows for storing temporary data (most commonly part of the CPU state) and later retrieving it. Instructions that interact with the stack are most notably “push” and “pop”, but also any instruction that use a subroutine, which is able to return via either “ret” or “iret”. Those instructions include “call” and “int”.
Unloaded Initial Executable	The unloaded initial executable is a binary copy of what the assembler produces when assembling from assembly code. It contains header information, a .data and a .text section without properly aligned addresses, so it won’t run directly when called (since it expects “wrong” offsets in memory). The aforementioned Program Loader is responsible for loading this program into memory and executing it.

.data and .text section default locations	Most of the time, the .data and .text sections of custom programs specify to be relocatable, meaning they run independent of their location in address space, once their instructions have been aligned (by the loader). These default locations are where the sections are loaded by default if they are relocatable. If the sections are not relocatable, they explicitly state an address to be loaded at, which the loader is responsible for loading them at this exact address.
[Reserved]	This memory region is reserved for later use. Do not use.

Tab. 3: Explanations for each memory region

2.5.1: MMIO

Memory-mapped input/output
- Video buffer x2 (inactive, active; RGB)
- Text only (output)
- Text input buffer (keyboard, serial)
- Interrupt data (also keyboard, but others too)

3.1: Instruction Encoding

Encoding Overview:
- Types (R/I/J)
- Encoding in binary of all three
- Example

3.2.1: Instructions Master Table

Table for general quick overview:
- Opcode, mnemonic, (usage) format

3.2.2: List of Instructions

For every instruction (not in a table):
Mnemonic
Format
Operation
Type
Opcode
Parameters
Encoding (e.g. `0x63af#### 0x########`)
Flags affected
Exceptions / interrupts / “traps”

4.3: Interrupts:

Interrupts are just special subroutines:
- They affect interrupt flags
- The instruction operand is not an address, but a vector (requires indirection via SHIT)

When interrupt is caused:
1. push SR to stack
2. "Interrupt enable" flag is cleared (set to 0)
3. "Interrupted" flag is set
4. push PC to stack
5. Extract interrupt vector from instruction operand, get address from vector, jump to vector address
6. ... ISR code runs ...
7. ISR eventually ends with the "iret" instruction
8. Pop PC from stack
9. Pop SR from stack
10. Resume execution at PC

→ Also check for double fault (when interrupted flag is already set before ISR start)

Hardware IRQs?

5.1: Notes, Limitations and Design Decisions regarding Emulation

[TODO: Focus more on why the CPU was designed this way because of limitations by emulation; Not what limitations come from the emulation process.]

Since at the current state, the CPU is neither implemented as true hardware, nor concrete enough to be programmable for hardware emulation devices (like FPGAs), there are several limiting factors considering the CPU itself, its environment, and the emulator host system.
First and foremost, the clock speed is currently limited to about 70 – 80 kHz depending on the scenario and host processing power. While this might seem fast enough for most cases, it might be sometimes required for higher speeds to be available. This is especially true for the long term, in which heavy use of screen buffers and live video updates might become necessary, while retaining acceptable latency.
Second, (and this is the primary reason the CPU architecture is limited to 32 bits) is that the CPU is able to handle up to 4 GiB of emulated RAM. This becomes a problem for the host, because without the necessary implementation considerations and parameters, most systems trying to emulate these spaces won’t be able to allocate this amount of host memory. This often reduces the effective size of emulated RAM, sometimes causing illegal address errors when trying to develop code for the CPU.